

Bug Severity Classification System

Low Level Design Document

Contents

1	Introduction	3
2	Database Schema	3
2.1	Table: users	3
2.2	Table: predictions	3
2.3	Severity Label Encoding	3
3	API Endpoint Specifications	4
3.1	Authentication Endpoints	4
3.1.1	POST /auth/signup	4
3.1.2	POST /auth/login	4
3.2	Prediction Endpoints	5
3.2.1	POST /predict	5
3.2.2	POST /feedback	6
3.3	History Endpoint	6
3.3.1	GET /history	6
3.4	Stats Endpoints	7
3.4.1	GET /feedback/stats	7
3.4.2	GET /admin/stats	7
3.4.3	GET /admin/users	7
3.5	Monitoring Endpoint	8
3.5.1	GET /metrics	8
3.6	Frontend Routes	8
4	ML Pipeline — Input/Output Specification	8
4.1	Stage 1: Preprocess (src/preprocess.py)	8
4.2	Stage 2: Transform (src/transform.py)	9
4.3	Stage 3: Train (src/train.py)	9
4.4	Stage 4: Evaluate (src/evaluate.py)	10
5	Drift Detection Specification (src/drift_detection.py)	11
5.1	Configuration Parameters	11
5.2	Check 1: Feedback Accuracy Drop	11
5.3	Check 2: KS Test — Distribution Drift	11
5.4	DriftResult Object	11
6	Airflow DAG Specification	12
6.1	Task Definitions	12
6.2	Trigger Rules	13
6.3	XCom Keys	13
7	Prometheus Metrics Specification	13
7.1	Counters	13
7.2	Gauges	13
7.3	Histograms	13
7.4	Summaries	14

8 Alert Rules Specification

14

1 Introduction

This document provides the low-level design specification for the Bug Severity Classification System. It defines the exact API endpoint contracts, database schema, ML pipeline input/output specifications, drift detection logic, and DAG task definitions. This document is intended for developers implementing or extending the system.

2 Database Schema

The system uses **PostgreSQL** as its primary database with the schema name `public`.

2.1 Table: users

Column	Type	Constraints	Description
id	INTEGER	PK, Auto-increment	Unique user identifier
username	VARCHAR	UNIQUE, NOT NULL	Login username
email	VARCHAR	UNIQUE, NOT NULL	User email address
password	VARCHAR	NOT NULL	Bcrypt hashed password
role	VARCHAR	NOT NULL	User role: admin or user
created_at	DATETIME	DEFAULT utcnow	Account creation timestamp

2.2 Table: predictions

Column	Type	Constraints	Description
id	INTEGER	PK, Auto-increment	Unique prediction ID
summary	VARCHAR	NOT NULL	Input bug summary text
predicted	VARCHAR	NOT NULL	Predicted severity label
confidence	FLOAT	NULLABLE	Model confidence score
probabilities	JSON	NULLABLE	Per-class probability dict
feedback_correct	INTEGER	NULLABLE	1 = correct, 0 = incorrect
ground_truth	VARCHAR	NULLABLE	User-provided correct label
user_id	INTEGER	FK → users.id	Submitting user reference
created_at	DATETIME	DEFAULT utcnow	Prediction timestamp

2.3 Severity Label Encoding

Labels are stored as strings in the predictions table and encoded as integers in the ML pipeline.

Label	Integer Encoding	Description
trivial	0	Cosmetic or negligible issue
minor	1	Small bug with workaround
normal	2	Standard bug requiring fix
major	3	Significant impact on functionality
critical	4	System crash or data loss

3 API Endpoint Specifications

All endpoints are served by FastAPI on **port 8000**. Authentication uses JWT Bearer tokens. Protected endpoints require the header:

```
Authorization: Bearer <token>
```

3.1 Authentication Endpoints

3.1.1 *POST /auth/signup*

Description: Register a new user account.

Authentication: Not required.

Request Body (JSON):

```
1 {
2   "username": "string",
3   "email": "string",
4   "password": "string",
5   "role": "string"    // "admin" or "user"
6 }
```

Response — 200 OK:

```
1 {
2   "msg": "Success"
3 }
```

Response — 400 Bad Request:

```
1 {
2   "detail": "Username taken"
3 }
```

Side Effects: Inserts a new row into users table. Updates bugs_signups_total Prometheus counter. Updates bugs_total_users_in_db Gauge.

3.1.2 *POST /auth/login*

Description: Authenticate a user and return a JWT token.

Authentication: Not required.

Request Body (JSON):

```
1 {
2   "username": "string",
3   "password": "string"
4 }
```

Response — 200 OK:

```
1 {
2   "token": "string",
3   "username": "string",
4   "role": "string"
5 }
```

Response — 400 Bad Request:

```
1 {
2   "detail": "Invalid credentials"
3 }
```

Side Effects: Updates `bugs_logins_total` counter (labeled success or failure). Records login latency in `bugs_login_latency_seconds` histogram.

3.2 Prediction Endpoints

3.2.1 *POST /predict*

Description: Submit a bug summary and receive a severity prediction.

Authentication: Required (JWT Bearer token).

Request Body (JSON):

```
1 {
2   "summary": "string"    // Bug description text
3 }
```

Response — 200 OK:

```
1 {
2   "prediction_id": "integer",
3   "severity": "string",    // trivial | minor | normal | major |
4     critical
5   "confidence": "float",   // 0.0 to 1.0
6   "probabilities": {
7     "trivial": "float",
8     "minor": "float",
9     "normal": "float",
10    "major": "float",
11    "critical": "float"
12  },
13  "drift_warning": "boolean"
14 }
```

Response — 401 Unauthorized:

```
1 {
2   "detail": "Not authenticated"
3 }
```

Side Effects: Inserts prediction into `predictions` table. Updates `bugs_predictions_total` counter (labeled by severity and `user_id`). Records latency in `bugs_prediction_latency_seconds`.

histogram. Updates `bugs_total_predictions_in_db` gauge. Observes confidence in `bugs_prediction_confidence` summary.

3.2.2 POST /feedback

Description: Submit feedback on a prediction (correct or incorrect).

Authentication: Not required.

Request Body (JSON):

```
1 {
2   "prediction_id": "integer",
3   "correct": "boolean",
4   "ground_truth": "string"    // Optional: actual severity label
5 }
```

Response — 200 OK:

```
1 {
2   "status": "ok"
3 }
```

Side Effects: Updates `feedback_correct` and `ground_truth` columns in `predictions` table. Updates `bugs_feedback_total` counter (labeled true or false). Records latency in `bugs_feedback_latency_seconds` histogram.

3.3 History Endpoint

3.3.1 GET /history

Description: Retrieve all predictions with usernames, ordered by most recent.

Authentication: Required (JWT Bearer token).

Request: No body. No query parameters.

Response — 200 OK:

```
1 [
2   {
3     "id": "integer",
4     "summary": "string",
5     "predicted": "string",
6     "confidence": "float",
7     "feedback_correct": "integer",    // 1, 0, or null
8     "ground_truth": "string",
9     "username": "string",
10    "created_at": "datetime"
11  }
12 ]
```

3.4 Stats Endpoints

3.4.1 GET /feedback/stats

Description: Return aggregated feedback accuracy statistics.

Authentication: Not required.

Response — 200 OK:

```
1 {
2   "total": "integer",
3   "correct": "integer",
4   "accuracy_pct": "float"    // Percentage, e.g. 87.5
5 }
```

3.4.2 GET /admin/stats

Description: Return system-wide statistics for admin users.

Authentication: Required. Role check enforced — admin only.

Response — 200 OK:

```
1 {
2   "total_predictions": "integer",
3   "total_users": "integer",
4   "total_feedback": "integer",
5   "drift_alerts": "integer",
6   "model_ready": "boolean",
7   "mlflow_url": "string"
8 }
```

3.4.3 GET /admin/users

Description: Return a list of all registered users.

Authentication: Required. Role check enforced — admin only.

Response — 200 OK:

```
1 [
2   {
3     "id": "integer",
4     "username": "string",
5     "email": "string",
6     "role": "string",
7     "created_at": "datetime"
8   }
9 ]
```


3.5 Monitoring Endpoint

3.5.1 GET /metrics

Description: Exposes Prometheus-compatible metrics in text format.

Authentication: Not required.

Response: Plain text in Prometheus exposition format. Scraped by Prometheus every 15 seconds.

3.6 Frontend Routes

Route	Method	Served File
/ or /login.html	GET	src/frontend/login.html
/index.html	GET	src/frontend/index.html
/history.html	GET	src/frontend/history.html
/admin.html	GET	src/frontend/admin.html

4 ML Pipeline — Input/Output Specification

4.1 Stage 1: Preprocess (`src/preprocess.py`)

Description: Loads raw CSV, cleans text, encodes labels, and splits into train/val/test sets.

Input:

Parameter	Source	Value
<code>--input</code>	Argument	<code>data/bugs.csv</code>
<code>columns_to_remove</code>	<code>params.yaml</code>	e.g. <code>[id, summary_len, summary_word_count]</code>
<code>text_column</code>	<code>params.yaml</code>	<code>summary</code>
<code>column_to_encode</code>	<code>params.yaml</code>	<code>severity</code>
<code>label_map</code>	<code>params.yaml</code>	<code>{trivial:0, minor:1, normal:2, major:3, cr}</code>
<code>test_size</code>	<code>params.yaml</code>	<code>0.2</code>
<code>random_state</code>	<code>params.yaml</code>	<code>42</code>

Processing Steps:

1. Drop columns listed in `columns_to_remove`
2. Lowercase and remove punctuation from `text_column` (summary)
3. Map severity string labels to integers using `label_map`
4. Stratified split: 80% train+val, 20% test
5. Further split train+val: 90% train, 10% val (relative)

Output:

File	Columns
data/processed/train.csv	summary, severity (integer)
data/processed/val.csv	summary, severity (integer)
data/processed/test.csv	summary, severity (integer)

4.2 Stage 2: Transform (**src/transform.py**)

Description: Generates sentence embeddings using all-MiniLM-L6-v2 (Sentence-Transformer) for each split.

Input:

File	Description
data/processed/train.csv	Preprocessed training data
data/processed/val.csv	Preprocessed validation data
data/processed/test.csv	Preprocessed test data

Processing Steps:

1. Load all-MiniLM-L6-v2 via SentenceTransformer
2. Generate 384-dimensional embeddings for each summary text
3. Attach embedding vectors as a new column to each DataFrame
4. Save each DataFrame as a Parquet file

Output:

File	Columns
data/features/train.parquet	id, summary, severity, embedding (list of 384 floats)
data/features/val.parquet	id, summary, severity, embedding
data/features/test.parquet	id, summary, severity, embedding

4.3 Stage 3: Train (**src/train.py**)

Description: Trains two sklearn models (Logistic Regression and Random Forest) and saves the best model by weighted F1 score.

Input:

Argument	Default	Description
--train-features	data/features/train.parquet	Training embeddings
--val-features	data/features/val.parquet	Validation embeddings
--model-output-path	models/best_model.pkl	Model save path
--metrics-path	metrics/best_model_metrics.json	Metrics save path

Models Trained:

Model	Key Parameters
Logistic Regression	<code>solver=saga, class_weight=balanced, warm_start=True</code>
Random Forest	<code>n_estimators=100, class_weight=balanced, warm_start=True</code>

Output:

File	Description
<code>models/best_model.pkl</code>	Pickle file: <code>{"model": <sklearn model>}</code>
<code>metrics/best_model_metrics.json</code>	<code>{"f1_weighted": float, "run_id": str}</code>
<code>metrics/train_distribution.json</code>	Per-label proportions from training set
<code>metrics/classification_report.txt</code>	sklearn classification report
<code>metrics/confusion_matrix.json</code>	Confusion matrix as nested list

MLflow Logging: All runs are logged under experiment `bug_severity_classification`. Logged metrics include `train_loss`, `val_loss`, `val_f1`, `val_acc`, `final_accuracy`, `final_f1_weighted`, `final_f1_macro`, and per-class F1 scores. Models are registered in the MLflow Model Registry.

4.4 Stage 4: Evaluate (`src/evaluate.py`)

Description: Evaluates the saved best model on the held-out test set and logs metrics to MLflow.

Input:

Argument	Default	Description
<code>--test-features</code>	required	Path to test embeddings parquet
<code>--model-path</code>	required	Path to <code>best_model.pkl</code>
<code>--metrics-path</code>	<code>metrics/test_metrics.json</code>	Output metrics path

Output — `metrics/test_metrics.json`:

```

1  {
2    "accuracy": 0.87,
3    "f1_weighted": 0.86,
4    "f1_macro": 0.84,
5    "per_class_f1": {
6      "trivial": 0.91,
7      "minor": 0.78,
8      "normal": 0.89,
9      "major": 0.83,
10     "critical": 0.92
11   },
12   "confusion_matrix": [[...], [...], [...], [...], [...]],
13   "n_test_samples": 1200
14 }
```

MLflow Logging: Logged under a run named `evaluate` in the same experiment.
 Metrics: `test_accuracy`, `test_f1_weighted`, `test_f1_macro`, per-class test F1 scores.
 Artifact: `test_metrics.json`.

5 Drift Detection Specification ([src/drift_detection.py](#))

5.1 Configuration Parameters

Parameter	Value	Description
<code>ACCURACY_THRESHOLD</code>	0.90	Feedback accuracy below this = drift
<code>KS_PVALUE_THRESHOLD</code>	0.05	KS p-value below this = drift
<code>MIN_FEEDBACK_COUNT</code>	10	Minimum feedbacks for accuracy check
<code>MIN_PRODUCTION_COUNT</code>	20	Minimum predictions for KS test

5.2 Check 1: Feedback Accuracy Drop

Input: Last 100 rows from `predictions` table where `feedback_correct` IS NOT NULL.

Logic:

$$accuracy = \frac{count(feedback_correct = 1)}{total\ feedbackrows}$$

Drift condition: `accuracy < 0.90`

5.3 Check 2: KS Test — Distribution Drift

Input:

- Production: last 200 predicted labels from `predictions` table (as integers)
- Training: label proportions from `metrics/train_distribution.json` (reconstructed as integer array)

Logic: Two-sample Kolmogorov-Smirnov test (`scipy.stats.ks_2samp`)

Drift condition: `p_value < 0.05`

5.4 DriftResult Object

Field	Type	Description
<code>drift_detected</code>	bool	True if either check fails
<code>reasons</code>	list[str]	Human-readable drift reasons
<code>details</code>	dict	Numeric values for all checks

Details dict keys: `feedback_total`, `feedback_accuracy`, `accuracy_drift`, `production_total`, `ks_stat`, `ks_pvalue`, `ks_drift`.

6 Airflow DAG Specification

DAG ID: bugs_severity_pipeline

Schedule: @daily (also supports manual trigger)

Start Date: 2026-04-27 UTC

Catchup: Disabled

6.1 Task Definitions

Task ID	Operator	Description
load_data	PythonOperator	Verifies data/bugs.csv exists and is readable
check_model_exists	BranchPythonOperator	Branches to load_production_data or preprocess
load_production_data	BranchPythonOperator	Queries DB; branches to check_drift or notify_no_drift
check_drift	PythonOperator	Runs drift detection; stores result in XCom
branch_on_drift	BranchPythonOperator	Reads XCom; routes to prometheus_alert or notify_no_drift
prometheus_alert	PythonOperator	POSTs drift alert to AlertManager REST API
append_feedback_to_csv	PythonOperator	Pulls feedback from DB; appends to data/bugs.csv
preprocess	PythonOperator	Runs src/preprocess.py via subprocess
transform	PythonOperator	Runs src/transform.py via subprocess
train	PythonOperator	Runs src/train.py via subprocess
evaluate	PythonOperator	Runs src/evaluate.py via subprocess
deploy	PythonOperator	Validates models/best_model.pkl is loadable
notify_success	PythonOperator	Sends success email with test metrics via Gmail SMTP
notify_no_drift	PythonOperator	Sends skip email with drift details via Gmail SMTP
end	EmptyOperator	Terminal task; accepts from success and no-drift paths

6.2 Trigger Rules

Task	Trigger Rule
preprocess	none_failed_min_one_success — accepts from first-train and retrain paths
notify_no_drift	none_failed_min_one_success — accepts from no-data and no-drift paths
end	none_failed_min_one_success — accepts from success and no-drift paths

6.3 XCom Keys

Key	Pushed by	Consumed by
drift_detected	check_drift	branch_on_drift
drift_reasons	check_drift	prometheus_alert, notify_success
drift_details	check_drift	notify_no_drift
production_count	load_production_data	notify_no_drift

7 Prometheus Metrics Specification

7.1 Counters

Metric Name	Labels	Description
bugs_predictions_total	severity, user_id	Total predictions made
bugs_logins_total	status	Total login attempts
bugs_signups_total	status	Total signup attempts
bugs_feedback_total	correct	Total feedback submissions
bugs_http_errors_total	endpoint, status_code	Total HTTP errors

7.2 Gauges

Metric Name	Description
bugs_active_requests	Currently active HTTP requests
bugs_total_users_in_db	Total registered users in DB
bugs_total_predictions_in_db	Total predictions stored in DB

7.3 Histograms

Metric Name	Labels	Buckets (seconds)
bugs_prediction_latency_seconds	user_id	0.01, 0.05, 0.1, 0.25, 0.5, 1.0, 2.0
bugs_feedback_latency_seconds	—	0.005, 0.01, 0.05, 0.1, 0.5
bugs_login_latency_seconds	—	0.01, 0.05, 0.1, 0.5

7.4 Summaries

Metric Name	Labels	Description
bugs_prediction_confidence	severity	Confidence score distribution. Auto-generates _su

8 Alert Rules Specification

Alert Name	Expression	For	Severity
BugsAppDown	up{job="bugs_app"} == 0	1m	critical
HighLoginFailureRate	increase(bugs_logins_total{status="failure"}[5m]) > 5	2m	warning
HighHTTPErrorRate	increase(bugs_http_errors_total[5m]) > 10	2m	warning
HighCPUUsage	CPU idle rate < 20%	2m	critical
HighMemoryUsage	RAM used > 85%	2m	warning
HighDiskUsage	Disk used > 90%	5m	critical
HighPredictionLatency	histogram_quantile(0.92, ...) > 2	2m	warning
NoPredictionsReceived	increase(bugs_predictions_total[15m]) == 0	15m	critical
TooManyActiveRequests	bugs_active_requests > 20	1m	warning
BugModelDriftDetected	Fired directly from Air-flow DAG via AlertManager REST API	—	warning